Compte rendu Projet Machine Learning

INGV - Volcanic Eruption Prediction

DA FONSECA Alexis & **ABDELLI** Khalid & **BENSALAH** Lockman

Étudiants en deuxième année de Master
Mathématiques et Applications à l'Université Gustave Eiffel

14 Mars 2025

TABLE DES MATIÈRES

I Présentation des données **3**

1 Organisation des données **4**
1.1 Description des fichiers de données 4

II Présentation du code **5**

2 Importance des librairies et modules utilisés **6**

3 Importation des données **7**
3.1 Explication des Étapes du Code 7
3.2 Finalisation de la Préparation des Données 8
3.3 Librairies Utilisées 8
3.4 Pourquoi cette approche ? 8

4 Entraînement du modèle **9**
4.1 Entraînement du Modèle 9
4.1.1 Optimisation des Hyperparamètres 9
4.1.2 Évaluation du Modèle 9
4.2 Librairies Utilisées 10
4.3 Pourquoi cette approche ? 10
4.4 Nos résultats d'entraînement 10

III Problèmes et Limites de notre étude **11**
4.5 Qualité des données insuffisante 12
4.6 Temps d'importation et consommation des ressources 12
4.7 Limites des résultats obtenus 12
4.8 Conclusion 12

IV Conclusion **13**

INTRODUCTION

Et si les scientifiques pouvaient prévoir les éruptions volcaniques avec la même précision que les prévisions météorologiques ? Bien que prévoir le temps plusieurs jours à l'avance soit un défi, les prévisions deviennent de plus en plus fiables à mesure que l'échéance se rapproche. Une approche similaire appliquée aux volcans pourrait faire une énorme différence : une seule éruption imprévue peut causer des dizaines de milliers de victimes. Si les scientifiques étaient capables d'anticiper avec précision la prochaine éruption, les évacuations pourraient être mieux organisées et les dégâts limités.

Aujourd'hui, les chercheurs estiment souvent le moment d'une éruption en surveillant les secousses volcaniques à travers les signaux sismiques. Sur certains volcans, cette activité augmente à l'approche d'une éruption. Cependant, interpréter ces schémas sismiques reste complexe. Dans les volcans les plus actifs, les techniques actuelles permettent parfois de prévoir une éruption quelques minutes à l'avance, mais elles restent inefficaces pour des prévisions à plus long terme.

L'Istituto Nazionale di Geofisica e Vulcanologia (INGV), en Italie, joue un rôle clé dans ce domaine. Spécialisé en géophysique et en volcanologie, il vise à mieux comprendre le fonctionnement de la Terre tout en réduisant les risques liés aux catastrophes naturelles. Chargé de surveiller en continu l'activité sismique et volcanique du pays, l'INGV cherche à identifier les premiers signaux annonciateurs d'une éruption afin d'en estimer le moment avec plus de précision.

Dans cette compétition, on mettra nos compétences en science des données à l'épreuve pour prédire la prochaine éruption d'un volcan. On analysera un vaste ensemble de données géophysiques recueillies par des capteurs placés sur des volcans actifs. Notre algorithme devra détecter des signaux caractéristiques dans les ondes sismiques permettant d'anticiper une éruption.

Première partie

Présentation des données

CHAPITRE 1

ORGANISATION DES DONNÉES

1.1 Description des fichiers de données

Pour la réalisation de notre projet, nous disposons de plusieurs fichiers de données essentiels :

— **train.csv** : Ce fichier contient les données d'entraînement utilisées pour modéliser le temps restant avant une éruption volcanique. Il comporte deux colonnes principales : `segment_id`, qui identifie chaque segment de données, et `time_to_eruption`, représentant le temps en secondes jusqu'à l'éruption.

— **sample_submission.csv** : Ce fichier sert d'exemple pour la soumission des résultats. Il présente la structure attendue pour les prédictions, avec les colonnes `segment_id` et `time_to_eruption`, où la seconde colonne est initialement remplie avec des valeurs nulles à remplacer par nos estimations.

— **Dossier train** : Ce dossier contient tous les fichiers csv qui contiennent les données issues de capteurs enregistrant différents phénomènes géophysiques. Dans chaque fichier, chaque ligne représente une mesure et les colonnes `sensor_1` à `sensor_10` correspondent aux valeurs enregistrées par 10 capteurs distincts.

— **Dossier test** : Ce dossier est construit de la même manière que le dossier train

Voici un exemple de graphe que l'on peut tracer avec nos données :

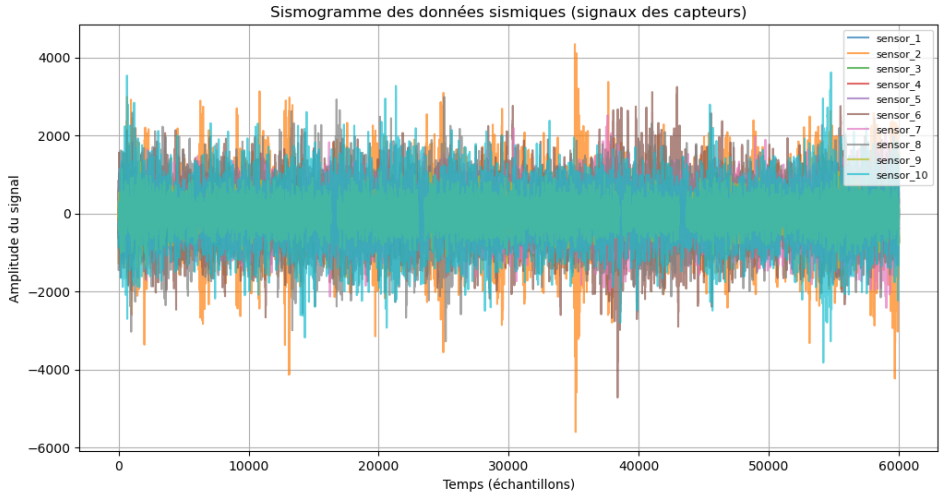


FIGURE 1.1 – Sismogramme des données sismiques

Deuxième partie

Présentation du code

CHAPITRE 2

IMPORTANCE DES LIBRAIRIES ET MODULES UTILISÉS

Le choix des modules Python en Machine Learning

Dans un projet de machine learning, le choix des modules Python est essentiel pour assurer la fiabilité et l'efficacité de l'étude. Chaque bibliothèque apporte des outils spécifiques facilitant la préparation des données, leur exploration et l'optimisation des modèles. Deux aspects clés doivent être pris en compte : la visualisation des données et l'ajustement des hyperparamètres.

Visualisation des données : un élément clé de l'analyse

Comprendre les données est une étape cruciale. Une exploration approfondie permet d'identifier des tendances, de détecter des anomalies et d'analyser la distribution des variables. Des bibliothèques comme matplotlib et seaborn permettent de représenter graphiquement les données et d'extraire des informations essentielles. Par exemple, les plots basiques peuvent révéler des relations entre variables, tandis que les histogrammes et les boîtes à moustaches aident à visualiser la distribution et les valeurs aberrantes. Ces visualisations orientent également le prétraitement des données.

Choix des hyperparamètres : optimiser les performances

La performance d'un modèle dépend fortement du choix de ses hyperparamètres. Un mauvais réglage peut entraîner du sur-apprentissage (overfitting) ou un modèle sous-performant. Des bibliothèques comme scikit-learn proposent des outils d'optimisation tels que GridSearchCV et RandomizedSearchCV, qui testent différentes configurations pour sélectionner les meilleures valeurs. Ajuster les hyperparamètres est donc essentiel pour obtenir un modèle performant et bien généralisé.

En conclusion, choisir judicieusement les modules Python est un facteur clé pour réussir un projet de machine learning. Une combinaison adaptée de bibliothèques dédiées à la visualisation et à l'optimisation garantit des analyses précises et des modèles efficaces. Ce choix impacte directement la qualité des résultats et la fiabilité des conclusions obtenues.

CHAPITRE 3

IMPORTATION DES DONNÉES

Dans ce code, plusieurs opérations sont réalisées pour extraire et structurer les données issues de fichiers CSV. Ces fichiers sont stockés dans un dossier et contiennent des informations segmentées par `segment_id`. L'objectif est de calculer plusieurs statistiques descriptives pour chaque segment.

3.1 Explication des Étapes du Code

- **Sélection d'un fichier de référence :**
  - Un fichier est choisi au hasard parmi les fichiers disponibles (en prenant le premier `segment_id` de la table `train`).
  - Il est chargé avec `pandas.read_csv()` pour identifier ses colonnes, en supposant que tous les fichiers partagent la même structure.
- **Initialisation d'un dictionnaire pour stocker les statistiques :**
  - Pour chaque colonne du fichier échantillon, des listes vides sont créées afin de stocker plusieurs statistiques : somme, moyenne, écart-type, valeurs extrêmes et quantiles.
- **Parcours des fichiers et extraction des statistiques :**
  - Chaque fichier correspondant à un `segment_id` est chargé.
  - Pour chaque colonne numérique, plusieurs statistiques sont calculées et stockées dans le dictionnaire :
    - `sum()` : Somme des valeurs
    - `mean()` : Moyenne
    - `std()` : Écart-type
    - `max()` / `min()` : Valeurs maximales et minimales
    - `quantile(0.90, 0.70, 0.50, 0.20, 0.10)` : Divers quantiles pour analyser la distribution des valeurs
  - Si une colonne est absente dans un fichier, elle est remplie avec des valeurs `NaN` pour garantir l'homogénéité des données.
- **Gestion des erreurs :**
  - En cas d'erreur de lecture d'un fichier, toutes les valeurs associées sont remplacées par `NaN` et un message d'erreur est affiché.
- **Ajustement des longueurs de listes :**
  - On s'assure que toutes les listes du dictionnaire ont la même longueur avant de convertir les données en un `DataFrame`.
- **Conversion en DataFrame :**
  - Toutes les données collectées sont transformées en un `pandas.DataFrame`, qui est affiché avec `head()` pour voir un aperçu des premières lignes.

3.2 Finalisation de la Préparation des Données

Une fois les mesures statistiques calculées, les données sont consolidées avec l'ensemble d'entraînement initial :

- **Fusion des données :**
  - Les statistiques calculées sont ajoutées au DataFrame `train` via la fonction `pd.concat()` pour obtenir `train_final`.
- **Suppression de la colonne `segment_id` :**
  - Comme cette colonne n'est pas utile pour l'apprentissage, elle est retirée afin de ne conserver que les variables pertinentes.
- **Séparation des variables explicatives et de la variable cible :**
  - Les caractéristiques du modèle (**features**) sont stockées dans `x`, obtenues en supprimant la colonne `time_to_eruption`.
  - La variable cible (**target**), qui représente le temps jusqu'à l'éruption, est extraite dans `y`.
- **Division en ensembles d'apprentissage et de test :**
  - Les données sont divisées en deux sous-ensembles :
    - **80% pour l'entraînement** du modèle.
    - **20% pour le test** afin d'évaluer les performances.
  - Cette séparation est réalisée grâce à la fonction `train_test_split()`.

Cette phase marque la fin de la préparation des données et prépare l'étape suivante : **l'apprentissage du modèle**. Les données étant correctement formatées et divisées, elles peuvent maintenant être utilisées pour entraîner un algorithme de machine learning.

3.3 Librairies Utilisées

- `pandas` :
  - Pour la manipulation des données (fusion des DataFrames, suppression de colonnes).
- `numpy` :
  - Pour les calculs statistiques et la gestion des valeurs manquantes.
- `sklearn.model_selection` :
  - Pour la fonction `train_test_split()` permettant la division des données.

3.4 Pourquoi cette approche ?

Ce processus assure une préparation rigoureuse des données avant l'apprentissage du modèle. Il permet une structuration propre des données et garantit que l'ensemble d'entraînement et de test sont bien définis, évitant ainsi les biais lors de l'évaluation du modèle.

CHAPITRE 4

ENTRAÎNEMENT DU MODÈLE

4.1 Entraînement du Modèle

L'entraînement du modèle est réalisé à l'aide de l'algorithme **XGBoost**, qui est un modèle de boosting performant pour la régression et la classification.

4.1.1 Optimisation des Hyperparamètres

— Un estimateur `XGBRegressor` est défini avec un objectif de minimisation de l'erreur quadratique.

— Une recherche aléatoire sur une grille d'hyperparamètres est effectuée avec `RandomizedSearchCV`, incluant :

  — **n_estimators** : Nombre d'arbres dans le modèle. Valeurs explorées : {100, 300, 500, 700}.

  — **learning_rate** : Taux d'apprentissage, contrôlant l'ampleur des mises à jour des poids. Valeurs testées : {0.01, 0.05, 0.1, 0.2}.

  — **max_depth** : Profondeur maximale des arbres. Valeurs possibles : {3, 5, 7, 9}.

  — **subsample** : Proportion d'échantillons utilisés pour chaque arbre, afin de réduire le surajustement. Valeurs considérées : {0.6, 0.8, 1.0}.

  — **colsample_bytree** : Fraction des caractéristiques utilisées pour chaque arbre. Valeurs évaluées : {0.6, 0.8, 1.0}.

  — **gamma** : Paramètre de régularisation contrôlant le gain minimum requis pour diviser un nœud. Valeurs testées : {0, 0.1, 0.2, 0.3}.

  — **min_child_weight** : Poids minimal requis pour créer une nouvelle division. Valeurs explorées : {1, 3, 5}.

  Une validation croisée avec `KFold` à 5 folds est utilisée pour évaluer les performances. Le modèle est entraîné avec la meilleure combinaison d'hyperparamètres trouvée.

4.1.2 Évaluation du Modèle

Une fois le modèle entraîné, il est évalué à l'aide de plusieurs métriques :

— **Coefficient de détermination (R^2) :** Évalue la qualité de la prédiction.

— **Erreur Absolue Moyenne (MAE) :** Moyenne des écarts absolus entre les valeurs prédites et réelles.

— **Erreur Quadratique Moyenne (RMSE) :** Racine carrée de la moyenne des erreurs au carré.

Les erreurs sont également converties en un format jour :heure :minute :seconde pour une meilleure interprétation des performances du modèle.

4.2 Librairies Utilisées

— `pandas :` Manipulation et gestion des données.

— `numpy :` Calculs numériques et gestion des valeurs manquantes.

— `sklearn.model_selection :` Séparation des données et validation croisée.

— `xgboost :` Entraînement du modèle XGBoost.

— `scikit-learn.metrics :` Évaluation des performances du modèle.

4.3 Pourquoi cette approche ?

L'utilisation de **XGBoost** combinée à une recherche d'hyperparamètres permet d'optimiser les performances du modèle. Les métriques d'évaluation fournissent une mesure claire des performances, facilitant ainsi l'analyse des résultats et l'amélioration du modèle si nécessaire.

4.4 Nos résultats d'entraînement

L'algorithme XGBoost a été optimisé à l'aide d'une validation croisée avec 6 folds sur un ensemble de 30 combinaisons d'hyperparamètres, soit un total de 150 ajustements. Les meilleurs paramètres obtenus sont : `subsample` = 1.0, `n_estimators` = 500, `min_child_weight` = 5, `max_depth` = 9, `learning_rate` = 0.1, `gamma` = 0.1 et `colsample_bytree` = 0.6.

L'évaluation des performances du modèle a été réalisée à l'aide de plusieurs métriques. Le meilleur score R^2 obtenu durant la validation croisée est de 0.7997. Sur l'ensemble de test, les performances du modèle sont les suivantes :

— **Erreur absolue moyenne (MAE)** : 44 days 07 :05 :45

— **Erreur quadratique moyenne (RMSE)** : 64 days 17 :14 :36.

— **Score R^2** : 0.82725

Ces résultats indiquent que le modèle fournit une bonne capacité de généralisation, avec un coefficient de détermination proche de 0.8. Toutefois, l'erreur en unités de temps reste significative, ce qui pourrait être amélioré par un prétraitement plus poussé des données ou une sélection plus fine des caractéristiques.

Troisième partie

Problèmes et Limites de notre étude

4.5 Qualité des données insuffisante

L'un des premiers défis que nous avons rencontrés concerne la qualité des données. En raison de valeurs manquantes, de données bruitées ou mal formatées, nous avons dû ajouter plusieurs mesures statistiques pour améliorer leur qualité. Parmi celles-ci :

— Calcul des **quantiles** pour apporter des informations supplémentaires.

— Calcul de grandeurs statistiques comme la **moyenne** et de l'**écart-type**.

Ces étapes étaient essentielles pour assurer la robustesse de notre modèle, mais elles ont nécessité un travail supplémentaire et un temps de calcul non négligeable.

4.6 Temps d'importation et consommation des ressources

L'importation des données a été un processus particulièrement long et gourmand en ressources. Nos fichiers de plus de 30GB ont mis à rude épreuve nos machines :

— Certains ordinateurs n'ont pas été capables d'exécuter le script en raison d'un manque de mémoire vive.

— Pour ceux qui y arrivaient, l'importation des données prenait un temps significatif, ralentissant l'ensemble du processus.

— L'entraînement de l'algorithme a également nécessité une puissance de calcul élevée, allongeant le temps d'exécution.

4.7 Limites des résultats obtenus

Malgré nos efforts, les résultats obtenus ne sont pas parfaits. Dans l'ensemble, notre modèle affiche une performance acceptable avec un coefficient de détermination (R^2) de **0.82**, ce qui est satisfaisant. Cependant :

— Certains résultats sont nettement moins précis que d'autres.

— Il subsiste une incertitude dans les prédictions, et notre modèle n'est pas fiable à 100%.

4.8 Conclusion

Les problèmes rencontrés, qu'ils concernent la qualité des données, la consommation des ressources ou la précision des résultats, illustrent bien les défis inhérents au Machine Learning. L'optimisation de la pipeline de traitement et une meilleure allocation des ressources seront des pistes d'amélioration pour nos futurs travaux.

Quatrième partie

Conclusion

CE QU'IL FAUT RETENIR

Dans ce projet, nous avons exploré l'utilisation du machine learning pour la prédiction des éruptions volcaniques en nous basant sur des données sismiques. Tout au long du travail, nous avons rencontré plusieurs défis, notamment en ce qui concerne la qualité des données, le temps d'importation, la consommation de ressources et l'interprétation des résultats obtenus.

Malgré ces obstacles, nous avons réussi à entraîner un modèle performant basé sur XGBoost, qui a permis d'obtenir un coefficient de détermination (R^2) de 0.82. Ce résultat est encourageant, bien que certaines prédictions restent incertaines. Il démontre que l'apprentissage automatique peut apporter des solutions prometteuses à la surveillance des volcans, mais qu'il nécessite encore des optimisations et des améliorations pour atteindre une précision plus fiable.

Pour aller plus loin, plusieurs pistes peuvent être envisagées :

— Affiner le prétraitement des données en explorant d'autres techniques de nettoyage et de sélection de caractéristiques.

— Tester d'autres modèles de machine learning, comme les réseaux neuronaux récurrents (RNN) ou les Transformers, qui pourraient mieux capturer les motifs temporels des signaux sismiques.

— Exploiter des ressources de calcul plus puissantes afin de réduire les temps d'entraînement et de permettre l'expérimentation de modèles plus complexes.

— Mettre en place un système de validation plus rigoureux pour mieux évaluer la robustesse du modèle sur des données non vues.

Ce projet nous a permis d'approfondir nos compétences en manipulation de données massives et en apprentissage supervisé.
Il illustre parfaitement les défis et les opportunités du machine learning dans le domaine de la gestion des risques naturels.